

✂ ДОМЕН 2: ПАМЯТЬ, ЖЕЛЕЗО И ПРОИЗВОДИТЕЛЬНОСТЬ — ПОЛНАЯ ДЕТАЛИЗАЦИЯ

СТРУКТУРА КАЖДОЙ ТЕМЫ

Для каждой темы я даю:

- **Что нужно знать** — теория для понимания
 - **Вопросы на собеседовании** (Junior → Senior → Staff)
 - **Практика** — конкретные задания для закрепления
-

РАЗДЕЛ 2.1: ОСНОВЫ ПАМЯТИ (JUNIOR → SENIOR)

2.1.1. Стек, куча и статическая память (Junior Level)

№ 2.1.1.1 — Стек (stack)

- **Что нужно знать:** LIFO, локальные переменные, адреса возврата функций, фрейм вызова (call frame)
- **Вопросы на собеседовании:** "Где хранятся локальные переменные в C++?"
- **Практика:** Написать программу, выводящую адреса локальных переменных в разных функциях, показать, как адреса увеличиваются/уменьшаются

№ 2.1.1.2 — Куча (heap)

- **Что нужно знать:** динамическое выделение через malloc/new, ручное управление памятью, фрагментация
- **Вопросы на собеседовании:** "В чем разница между стеком и кучей?"
- **Практика:** Написать программу, выделяющую память через new int[100], показать адрес и объяснить, где он находится

№ 2.1.1.3 — Статическая память

- **Что нужно знать:** глобальные переменные, static переменные внутри функций
- **Вопросы на собеседовании:** "Где хранятся глобальные переменные?"
- **Практика:** Написать программу, демонстрирующую время жизни глобальной переменной, сравнить адреса с локальными переменными

№ 2.1.1.4 — Размер стека

- **Что нужно знать:** по умолчанию 1-8 МБ (Windows/Linux), переполнение стека (stack overflow)
- **Вопросы на собеседовании:** "Что такое stack overflow? Как его избежать?"
- **Практика:** Написать рекурсивную функцию без базового случая и вызвать stack overflow, посмотреть ошибку

№ 2.1.1.5 — Выравнивание (alignment)

- **Что нужно знать:** структуры в памяти, padding, alignas (C++), StructLayout (C#)
- **Вопросы на собеседовании:** "Почему структуры занимают больше места, чем сумма полей?"
- **Практика:** Написать структуры с разным порядком полей, вывести sizeof и объяснить разницу

№ 2.1.1.6 — Жизненный цикл объектов

- **Что нужно знать:** конструктор → использование → деструктор (C++), GC (C#)
- **Вопросы на собеседовании:** "Когда вызывается деструктор в C++? Когда объект удаляется в C#?"
- **Практика:** Написать класс с конструктором/деструктором и проследить моменты их вызова

2.1.2. Представление данных в памяти (Junior → Middle Level)

№ 2.1.2.1 — Целые числа

- **Что нужно знать:** Two's complement, знаковые vs беззнаковые, переполнение (overflow)
- **Вопросы на собеседовании:** "Что произойдет при переполнении знакового int?"
- **Практика:** Написать программу, демонстрирующую переполнение INT_MAX + 1, объяснить результат

№ 2.1.2.2 — Порядок байт (Endianness)

- **Что нужно знать:** Big Endian vs Little Endian, как проверить на своей машине

- **Вопросы на собеседовании:** "Как определить порядок байт на системе?"
- **Практика:** Написать программу, проверяющую порядок байт через union или приведение указателей

№ 2.1.2.3 — IEEE 754

- **Что нужно знать:** знак (1 бит), экспонента (8 бит), мантисса (23 бита) для float; double (11/52)
- **Вопросы на собеседовании:** "Почему $0.1 + 0.2 \neq 0.3$?"
- **Практика:** Вывести побайтово число 3.14159f, объяснить каждый байт (знак, экспонента, мантисса)

№ 2.1.2.4 — Специальные значения

- **Что нужно знать:** +inf, -inf, NaN (Not a Number), денормализованные числа
- **Вопросы на собеседовании:** "Что такое NaN в IEEE 754?"
- **Практика:** Сгенерировать NaN, inf, -inf и проверить их через isnan(), isinf()

№ 2.1.2.5 — Представление указателей

- **Что нужно знать:** размер (8 байт на 64-bit), NULL/nullptr (0)
- **Вопросы на собеседовании:** "Сколько байт занимает указатель на 64-bit системе?"
- **Практика:** Вывести sizeof(void*), показать адрес указателя

№ 2.1.2.6 — Объектная модель C++

- **Что нужно знать:** расположение vtable (виртуальная таблица) в памяти, множественное наследование
- **Вопросы на собеседовании:** "Где находится vtable в объекте с виртуальными функциями?"
- **Практика:** Создать класс с виртуальными функциями, вывести адрес объекта и адрес vtable

2.1.3. Экспертный уровень (Senior / Staff Level)

№ 2.1.3.1 — Аллокаторы памяти

- **Что нужно знать:** malloc (ptmalloc2, jemalloc, tcmalloc), системные вызовы (brk, mmap)
- **Вопросы на собеседовании:** "Как malloc выделяет память? В чем разница между brk и mmap?"
- **Практика:** Написать программу, выделяющую разные объемы памяти, и отследить системные вызовы через strace

№ 2.1.3.2 — Фрагментация памяти

- **Что нужно знать:** внешняя и внутренняя фрагментация, как с ней борются аллокаторы
- **Вопросы на собеседовании:** "Что такое фрагментация памяти и почему она плоха?"
- **Практика:** Написать программу, создающую фрагментацию (выделение/освобождение блоков разного размера)

№ 2.1.3.3 — Пул памяти (Memory Pool)

- **Что нужно знать:** кастомные аллокаторы для уменьшения фрагментации и ускорения выделения
- **Вопросы на собеседовании:** "Как реализовать пул памяти для частого выделения объектов одного размера?"
- **Практика:** Реализовать пул объектов (Object Pool) с кастомным аллокатором на C++

№ 2.1.3.4 — Арендный аллокатор (Arena)

- **Что нужно знать:** линейное выделение, быстрый reset, использование в игровых движках
- **Вопросы на собеседовании:** "Что такое Arena аллокатор и когда его использовать?"
- **Практика:** Реализовать линейный аллокатор с методами allocate() и reset()

№ 2.1.3.5 — Stack allocator

- **Что нужно знать:** выделение памяти из стека (alloca), использование для быстрых временных объектов
- **Вопросы на собеседовании:** "Что такое alloca? Какие у нее ограничения?"
- **Практика:** Написать программу с alloca, сравнить скорость с malloc

№ 2.1.3.6 — Память в C#

- **Что нужно знать:** managed heap, pinned objects (закрепка), внутренности GC
 - **Вопросы на собеседовании:** "Что такое pinned объект в C#? Зачем он нужен?"
 - **Практика:** Написать программу, использующую GCHandle.Alloc с GCHandleType.Pinned
-

2.1.4. Практические задания (сквозные)

№ 2.1.4.1

- **Задание:** Реализовать свой аллокатор памяти (пул объектов) для класса Widget
- **Что проверяет:** Понимание управления памятью

№ 2.1.4.2

- **Задание:** Написать программу, демонстрирующую утечку памяти, и найти ее с помощью Valgrind/AddressSanitizer
- **Что проверяет:** Навыки отладки памяти

№ 2.1.4.3

- **Задание:** Реализовать линейный аллокатор (Arena) и измерить скорость выделения по сравнению с malloc
- **Что проверяет:** Понимание производительности аллокаторов

№ 2.1.4.4

- **Задание:** Написать программу, определяющую порядок байт на системе через union
- **Что проверяет:** Знание представления данных

№ 2.1.4.5

- **Задание:** Реализовать класс shared_ptr с подсчетом ссылок (без использования стандартных)
 - **Что проверяет:** Понимание умных указателей и управления памятью
-

РАЗДЕЛ 2.2: КЭШИ, ВИРТУАЛЬНАЯ ПАМЯТЬ И ПРОЦЕССОР (JUNIOR → SENIOR)

2.2.1. Кэш-память (Junior → Middle Level)

№ 2.2.1.1 — Иерархия памяти

- **Что нужно знать:** регистры → L1 (32KB) → L2 (256KB) → L3 (2-16MB) → RAM → SSD/HDD
- **Вопросы на собеседовании:** "Какая иерархия памяти в современном процессоре?"
- **Практика:** Нарисовать схему иерархии с указанием размеров и латентности

№ 2.2.1.2 — Кэш-линии

- **Что нужно знать:** размер обычно 64 байта, чтение/запись блоками
- **Вопросы на собеседовании:** "Что такое кэш-линия?"
- **Практика:** Написать программу, которая читает память с шагом 64 байта и замеряет скорость (cache hit) vs шагом 4096 байт (cache miss)

№ 2.2.1.3 — Принцип локальности

- **Что нужно знать:** пространственная (близкие адреса) и временная (повторное использование)
- **Вопросы на собеседовании:** "Почему обход массива по строкам быстрее, чем по столбцам?"
- **Практика:** Написать бенчмарк обхода матрицы 10000×10000 по строкам и по столбцам

№ 2.2.1.4 — Cache Miss

- **Что нужно знать:** холодный (первые данные), конфликтный, емкостный
- **Вопросы на собеседовании:** "Какие виды cache miss бывают?"
- **Практика:** Написать программу, демонстрирующую конфликтный кэш-промах

№ 2.2.1.5 — False Sharing (ложное разделение)

- **Что нужно знать:** когда два потока пишут в разные переменные одной кэш-линии

- **Вопросы на собеседовании:** "Что такое False Sharing? Как его обнаружить?"
- **Практика:** Написать бенчмарк с двумя потоками, инкрементирующими соседние переменные, замерить время, исправить через alignas(64)

№ 2.2.1.6 — Prefetching

- **Что нужно знать:** аппаратный и программный (`__builtin_prefetch` в C++)
 - **Вопросы на собеседовании:** "Как работает программный prefetch?"
 - **Практика:** Написать программу с `__builtin_prefetch`, замерить ускорение
-

2.2.2. Виртуальная память (Middle Level)

№ 2.2.2.1 — Виртуальная память

- **Что нужно знать:** страницы (обычно 4KB), таблицы страниц, MMU (Memory Management Unit)
- **Вопросы на собеседовании:** "Что такое виртуальная память? Как она работает?"
- **Практика:** Написать программу, выделяющую память, и отследить с помощью `rmap` или `/proc/pid/maps`

№ 2.2.2.2 — TLB (Translation Lookaside Buffer)

- **Что нужно знать:** кэш для таблиц страниц, TLB miss (дорогой)
- **Вопросы на собеседовании:** "Что такое TLB и почему он важен?"
- **Практика:** Написать программу, обходящую память с большим шагом (2MB) и замерить время — показать эффект TLB miss

№ 2.2.2.3 — Page Fault

- **Что нужно знать:** обработка, swapping, demand paging
- **Вопросы на собеседовании:** "Что происходит при Page Fault?"
- **Практика:** Написать программу, выделяющую память и не использующую ее, посмотреть, когда происходит выделение физической памяти

№ 2.2.2.4 — Размер страницы

- **Что нужно знать:** 4KB (стандарт), огромные страницы (Huge Pages — 2MB/1GB) для производительности
- **Вопросы на собеседовании:** "Как использовать Huge Pages для ускорения приложения?"
- **Практика:** Настроить Huge Pages на Linux и замерить ускорение для приложения с большим объемом данных

№ 2.2.2.5 — Memory Mapping

- **Что нужно знать:** mmap (Linux), отображение файлов в память
- **Вопросы на собеседовании:** "Что такое mmap и когда его использовать?"
- **Практика:** Написать программу, читающую файл через mmap и через read, сравнить скорость

№ 2.2.2.6 — Swap

- **Что нужно знать:** подкачка на диск, как влияет на производительность
- **Вопросы на собеседовании:** "Когда используется swap? Как это влияет на производительность?"
- **Практика:** Включить swap, выделить больше памяти, чем есть физической, и замерить падение производительности

2.2.3. Экспертный уровень (Senior / Staff Level)

№ 2.2.3.1 — NUMA (Non-Uniform Memory Access)

- **Что нужно знать:** доступ к памяти через разные контроллеры, локальная/удаленная память
- **Вопросы на собеседовании:** "Как NUMA влияет на производительность многопоточных приложений?"
- **Практика:** Написать программу, выделяющую память на разных NUMA-нодах, замерить латентность, использовать numactl

№ 2.2.3.2 — Оптимизация кэша

- **Что нужно знать:** структуры данных с учетом кэша (cache-friendly), Linked List vs Array
- **Вопросы на собеседовании:** "Почему std::vector часто быстрее std::list?"

- **Практика:** Написать бенчмарк для `std::vector` и `std::list` при обходе и сравнить количество `cache miss`

№ 2.2.3.3 — Битность

- **Что нужно знать:** 32-bit vs 64-bit, ограничения адресного пространства
- **Вопросы на собеседовании:** "Какие ограничения у 32-битного приложения?"
- **Практика:** Собрать программу как 32-bit, выделить >2GB памяти и показать ошибку

№ 2.2.3.4 — Атомарные операции

- **Что нужно знать:** CAS (Compare-And-Swap), LL/SC (Load-Link/Store-Conditional)
- **Вопросы на собеседовании:** "Как работают атомарные операции на уровне процессора?"
- **Практика:** Написать программу с `std::atomic`, дизассемблировать и показать инструкции (`lock cmpxchg`)

№ 2.2.3.5 — Barriers

- **Что нужно знать:** `mfence`, `lfence`, `sfence`, `memory_order` в C++ (`acquire/release/seq_cst`)
- **Вопросы на собеседовании:** "Что такое барьеры памяти? Зачем они нужны?"
- **Практика:** Написать программу с `memory_order`, показать, как барьер влияет на переупорядочивание операций

№ 2.2.3.6 — Профилирование `cache miss`

- **Что нужно знать:** `perf` (Linux), `intel VTune`, `dotnet-counters` для C#
- **Вопросы на собеседовании:** "Как измерить количество `cache miss` в программе?"
- **Практика:** Запустить `perf stat -e cache-misses,cache-references` для своей программы

2.2.4. Практические задания (сквозные)

№ 2.2.4.1

- **Задание:** Написать бенчмарк, демонстрирующий разницу между обходом матрицы по строкам и столбцам
- **Что проверяет:** Понимание кэша

№ 2.2.4.2

- **Задание:** Написать программу с False Sharing, замерить время, исправить через alignas(64)
- **Что проверяет:** Понимание кэш-линий и многопоточности

№ 2.2.4.3

- **Задание:** Написать программу, использующую mmap для чтения большого файла, сравнить с fread
- **Что проверяет:** Знание работы с памятью

№ 2.2.4.4

- **Задание:** Настроить Huge Pages и замерить ускорение для приложения
- **Что проверяет:** Понимание управления памятью

№ 2.2.4.5

- **Задание:** Написать lock-free структуру с использованием атомарных операций (CAS)
- **Что проверяет:** Понимание атомарных операций и многопоточности

РАЗДЕЛ 2.3: МНОГОПОТОЧНОСТЬ И СИНХРОНИЗАЦИЯ (JUNIOR → SENIOR)

2.3.1. Основы многопоточности (Junior Level)

№ 2.3.1.1 — Потоки (threads)

- **Что нужно знать:** std::thread (C++), Thread (C#), создание, запуск, join()/detach()
- **Вопросы на собеседовании:** "Как создать поток в C++?"
- **Практика:** Написать программу с 5 потоками, каждый выводит свой ID

№ 2.3.1.2 — Состояния потока

- **Что нужно знать:** Ready, Running, Blocked, Terminated
- **Вопросы на собеседовании:** "Какие состояния бывают у потока?"

- **Практика:** Написать программу, демонстрирующую переход между состояниями

№ 2.3.1.3 — Гонка данных (Data Race)

- **Что нужно знать:** несколько потоков пишут в одну переменную
- **Вопросы на собеседовании:** "Что такое data race?"
- **Практика:** Написать программу с 10 потоками, инкрементирующими глобальную переменную без синхронизации, показать неправильный результат

№ 2.3.1.4 — Мьютексы

- **Что нужно знать:** `std::mutex`, `lock()/unlock()`, `std::lock_guard` (C++), `lock` (C#)
- **Вопросы на собеседовании:** "Как синхронизировать доступ к общим данным?"
- **Практика:** Написать программу с мьютексом, исправить data race

№ 2.3.1.5 — Deadlock

- **Что нужно знать:** взаимная блокировка, условия возникновения (взаимное исключение, удержание, циклическое ожидание)
- **Вопросы на собеседовании:** "Что такое deadlock? Как его избежать?"
- **Практика:** Написать программу с двумя мьютексами и создать deadlock

№ 2.3.1.6 — Condition Variable

- **Что нужно знать:** `std::condition_variable` (C++), `Monitor.Wait/Pulse` (C#)
- **Вопросы на собеседовании:** "Как реализовать producer-consumer с помощью condition variable?"
- **Практика:** Реализовать producer-consumer с `std::condition_variable`

2.3.2. Продвинутая синхронизация (Middle Level)

№ 2.3.2.1 — Read-Write Lock

- **Что нужно знать:** `std::shared_mutex` (C++14/17), чтение (shared) vs запись (exclusive)
- **Вопросы на собеседовании:** "Когда использовать read-write lock?"

- **Практика:** Реализовать кеш с read-write lock (много читателей, один писатель)

№ 2.3.2.2 — Спинлок (Spinlock)

- **Что нужно знать:** активное ожидание, использование для коротких критических секций
- **Вопросы на собеседовании:** "Когда использовать спинлок вместо мьютекса?"
- **Практика:** Реализовать спинлок на `std::atomic_flag` и сравнить с мьютексом на бенчмарке

№ 2.3.2.3 — Semaphore

- **Что нужно знать:** семафоры (не в стандартном C++ до C++20, в C# Semaphore), счетчик
- **Вопросы на собеседовании:** "Что такое семафор? Чем отличается от мьютекса?"
- **Практика:** Реализовать семафор с помощью `std::condition_variable`

№ 2.3.2.4 — Barrier

- **Что нужно знать:** `std::barrier` (C++20), точка синхронизации для группы потоков
- **Вопросы на собеседовании:** "Что такое барьер и когда он используется?"
- **Практика:** Написать программу, где 5 потоков достигают барьера и продолжают работу вместе

№ 2.3.2.5 — Атомарные операции

- **Что нужно знать:** `std::atomic<int>` (C++), Interlocked (C#), инкремент без блокировок
- **Вопросы на собеседовании:** "Как реализовать счетчик без мьютекса?"
- **Практика:** Реализовать счетчик с `std::atomic<int>` и сравнить со счетчиком с мьютексом

№ 2.3.2.6 — Thread Pool

- **Что нужно знать:** пул потоков для переиспользования, уменьшение накладных расходов
- **Вопросы на собеседовании:** "Зачем нужен пул потоков?"

- **Практика:** Реализовать свой пул потоков с очередью задач
-

2.3.3. Экспертный уровень (Senior / Staff Level)

№ 2.3.3.1 — Lock-free структуры

- **Что нужно знать:** очередь (Michael-Scott), стек (Treiber), без мьютексов
- **Вопросы на собеседовании:** "Как реализовать потокобезопасную очередь без блокировок?"
- **Практика:** Реализовать lock-free стек (Treiber) с использованием CAS

№ 2.3.3.2 — Memory ordering

- **Что нужно знать:** memory_order_relaxed, memory_order_acquire, memory_order_release, memory_order_seq_cst
- **Вопросы на собеседовании:** "В чем разница между memory_order_relaxed и memory_order_seq_cst?"
- **Практика:** Написать программу с разными memory_order и показать, как они влияют на видимость данных

№ 2.3.3.3 — Happens-before relationship

- **Что нужно знать:** формальная модель памяти
- **Вопросы на собеседовании:** "Что такое happens-before в C++ memory model?"
- **Практика:** Изучить формальную модель памяти C++ и написать примеры

№ 2.3.3.4 — ABA проблема

- **Что нужно знать:** проблема в lock-free структурах, решение с использованием счетчика версий
- **Вопросы на собеседовании:** "Что такое ABA проблема и как ее решить?"
- **Практика:** Изучить проблему ABA в lock-free списке и показать пример

№ 2.3.3.5 — Fork-Join модель

- **Что нужно знать:** std::async, std::future, std::promise

- **Вопросы на собеседовании:** "Как работает `std::async`?"
- **Практика:** Написать программу, использующую `std::async` для параллельной обработки данных

№ 2.3.3.6 — Планировщики потоков

- **Что нужно знать:** как планируются потоки в ОС (Linux CFS, Windows), приоритеты
- **Вопросы на собеседовании:** "Как планируются потоки в Linux?"
- **Практика:** Написать программу с разными приоритетами потоков и показать разницу

№ 2.3.3.7 — Асинхронность в C#

- **Что нужно знать:** `async/await`, Task, State Machine
- **Вопросы на собеседовании:** "Как работает `async/await` под капотом?"
- **Практика:** Дизассемблировать асинхронный метод в IL и показать state machine

№ 2.3.3.8 — Producer-Consumer с Zero-Copy

- **Что нужно знать:** использование кольцевого буфера (ring buffer), алгоритм Disruptor
- **Вопросы на собеседовании:** "Как реализовать высокопроизводительный producer-consumer без копирования данных?"
- **Практика:** Реализовать кольцевой буфер (circular buffer) для producer-consumer

2.3.4. Практические задания (сквозные)

№ 2.3.4.1

- **Задание:** Реализовать producer-consumer с использованием `std::condition_variable`
- **Что проверяет:** Знание синхронизации

№ 2.3.4.2

- **Задание:** Реализовать свой thread pool с очередью задач
- **Что проверяет:** Понимание пулов потоков

№ 2.3.4.3

- **Задание:** Реализовать lock-free стек (Treiber) на C++
- **Что проверяет:** Понимание lock-free структур

№ 2.3.4.4

- **Задание:** Реализовать блокирующую очередь с timeout на C++
- **Что проверяет:** Знание condition_variable и таймаутов

№ 2.3.4.5

- **Задание:** Написать бенчмарк для сравнения производительности мьютекса, спинлока и атомарного инкремента
- **Что проверяет:** Понимание производительности синхронизации

№ 2.3.4.6

- **Задание:** Реализовать async/await в C++ с помощью корутин (C++20)
- **Что проверяет:** Понимание корутин и асинхронности

РАЗДЕЛ 2.4: ОПТИМИЗАЦИЯ И ПРОФИЛИРОВАНИЕ (SENIOR → STAFF)

2.4.1. Профилирование (Senior Level)

№ 2.4.1.1 — Инструменты профилирования

- **Что нужно знать:** perf (Linux), Intel VTune, dotnet-trace, dotnet-dump, Visual Studio Profiler
- **Вопросы на собеседовании:** "Как найти узкое место в производительности программы?"
- **Практика:** Запустить perf record для своей программы, проанализировать perf report

№ 2.4.1.2 — CPU Profiling

- **Что нужно знать:** sampling vs instrumentation, накладные расходы
- **Вопросы на собеседовании:** "В чем разница между sampling и instrumentation профилированием?"
- **Практика:** Сравнить perf (sampling) и Valgrind (instrumentation) на примере

№ 2.4.1.3 — Memory Profiling

- **Что нужно знать:** dotnet-gcdump (C#), massif (Valgrind), heaptrack

- **Вопросы на собеседовании:** "Как найти утечку памяти в C# приложениях?"
- **Практика:** Снять dotnet-gcdump, проанализировать поколения и LOH

№ 2.4.1.4 — I/O Profiling

- **Что нужно знать:** iostat, iotop, blktrace
- **Вопросы на собеседовании:** "Как измерить скорость чтения с диска?"
- **Практика:** Запустить iostat -x 1 и проанализировать нагрузку на диск

№ 2.4.1.5 — Network Profiling

- **Что нужно знать:** tcpdump, wireshark, nethogs
- **Вопросы на собеседовании:** "Как проанализировать сетевой трафик?"
- **Практика:** Запустить tcpdump для порта 80 и проанализировать пакеты

№ 2.4.1.6 — CPU Cache Miss анализ

- **Что нужно знать:** perf stat -e cache-misses,cache-references
- **Вопросы на собеседовании:** "Как измерить количество кэш-промахов?"
- **Практика:** Запустить perf stat -e cache-misses,cache-references для своей программы и оптимизировать

2.4.2. Оптимизация кода (Staff Level)

№ 2.4.2.1 — Оптимизация циклов

- **Что нужно знать:** развертка циклов (loop unrolling), префетчинг, устранение зависимостей
- **Вопросы на собеседовании:** "Как оптимизировать цикл для работы с большими массивами?"
- **Практика:** Написать программу, демонстрирующую ускорение от развертки цикла

№ 2.4.2.2 — Инлайн (inlining)

- **Что нужно знать:** inline в C++,MethodImplOptions.AggressiveInlining в C#, плюсы и минусы

- **Вопросы на собеседовании:** "Что такое инлайн? Когда его использовать?"
- **Практика:** Написать программу с `__attribute__((always_inline))` и показать ускорение

№ 2.4.2.3 — SIMD (Single Instruction, Multiple Data)

- **Что нужно знать:** AVX, SSE, intrinsics
- **Вопросы на собеседовании:** "Как использовать SIMD для ускорения вычислений?"
- **Практика:** Написать программу с AVX-интринсиками для сложения векторов

№ 2.4.2.4 — Branch Prediction

- **Что нужно знать:** предсказание переходов, влияние на производительность
- **Вопросы на собеседовании:** "Как работает предсказание переходов?"
- **Практика:** Написать программу с неупорядоченными данными (сортированными/не сортированными) и показать разницу

№ 2.4.2.5 — Оптимизация памяти

- **Что нужно знать:** использование пулов объектов, избегание аллокаций в циклах, stack allocation
- **Вопросы на собеседовании:** "Как избежать аллокаций в критическом по производительности коде?"
- **Практика:** Переписать код, использующий `new` в цикле, на использование `std::array` или пула

№ 2.4.2.6 — Оптимизация компилятора

- **Что нужно знать:** -O2 vs -O3, -march=native, LTO (Link Time Optimization)
- **Вопросы на собеседовании:** "Какие флаги оптимизации нужно использовать для C++ проекта?"
- **Практика:** Собрать проект с разными флагами оптимизации и сравнить производительность

№ 2.4.2.7 — Оптимизация кэша

- **Что нужно знать:** предотвращение False Sharing, правильное выравнивание, использование `alignas`

- **Вопросы на собеседовании:** "Как структурировать данные, чтобы уменьшить количество cache miss?"
- **Практика:** Переписать структуру данных с учетом кэша и показать ускорение

№ 2.4.2.8 — Оптимизация строк

- **Что нужно знать:** `std::string` vs `std::string_view` (C++17), `StringBuilder` (C#)
 - **Вопросы на собеседовании:** "Как эффективно работать со строками в C++ и C#?"
 - **Практика:** Написать бенчмарк для конкатенации строк с `std::string`, `std::string_view` и `StringBuilder`
-

2.4.3. Практические задания (сквозные)

№ 2.4.3.1

- **Задание:** Написать бенчмарк для сравнения производительности `std::vector` и `std::list` при вставке/удалении
- **Что проверяет:** Понимание cache-friendly структур

№ 2.4.3.2

- **Задание:** Профилировать программу с `perf`, найти самую медленную функцию, оптимизировать
- **Что проверяет:** Навыки профилирования

№ 2.4.3.3

- **Задание:** Написать программу с SIMD-интринсиками для умножения матриц, сравнить с обычным кодом
- **Что проверяет:** Понимание SIMD

№ 2.4.3.4

- **Задание:** Оптимизировать программу с False Sharing, показать ускорение
- **Что проверяет:** Понимание кэша

№ 2.4.3.5

- **Задание:** Написать benchmark для сравнения `async/await` и `Task` в C#

- **Что проверяет:** Понимание асинхронности
-

☞ СВОДНЫЙ ЧЕК-ЛИСТ ДЛЯ ПОДГОТОВКИ К СОБЕСЕДОВАНИЮ

Junior Level (должен знать 100%)

Стек vs Куча: разница, где что хранится

Примитивные типы: размеры, представление (целые, float)

Выравнивание структур (padding)

Основы многопоточности: создание потоков, мьютексы, data race

Deadlock: что это, как избежать

Базовое профилирование: perf stat

Middle Level (должен знать 100%)

IEEE 754: структура float/double, NaN, Inf

Кэш: иерархия, кэш-линии, False Sharing

Виртуальная память: страницы, TLB, Page Fault

Продвинутая синхронизация: read-write lock, spinlock, condition variable

std::atomic, атомарные операции

mmap, malloc vs new

Профилирование: perf, dotnet-trace

Senior Level (должен знать 100%)

Аллокатеры: пул памяти, арена, jemalloc, tcmalloc

NUMA, Huge Pages

Lock-free структуры (стек, очередь)

Memory ordering (acquire/release/seq_cst)

ABA проблема и ее решение

Оптимизация: SIMD, инлайн, branch prediction

Профилирование: cache miss, TLB miss

Staff Level (должен знать 100%)

Внутреннее устройство аллокаторов (ptmalloc, tcmalloc)

Формальная модель памяти (happens-before)

Разработка собственных lock-free структур

Оптимизация под конкретную архитектуру (x86 vs ARM)

Оптимизация компилятора (LTO, PGO)

Системное профилирование (perf, eBPF)

🔨 ПРАКТИЧЕСКИЙ ПРОЕКТ ДЛЯ ЗАКРЕПЛЕНИЯ ДОМЕНА 2

Задание: Реализовать высокопроизводительный кеш (in-memory cache) с поддержкой многопоточности.

Шаги:

1. Реализовать хеш-таблицу с открытой адресацией (для кеша)
2. Добавить поддержку TTL (Time-To-Live) для записей
3. Реализовать политику вытеснения LRU (Least Recently Used) с использованием двусвязного списка
4. Реализовать потокобезопасность:
 - Чтение без блокировок (атомарные операции)
 - Запись с блокировкой на уровне корзины
 - Или lock-free реализация для чтения
5. Добавить профилирование:
 - Измерить cache miss
 - Измерить время доступа
 - Показать масштабируемость с количеством потоков

Проверка:

- Кеш должен работать в 1000 раз быстрее, чем обращение к БД
- Должен корректно работать при 10+ потоках
- Не должно быть утечек памяти